Introduction to Data Structures and Algorithms

Data Structure is a way of collecting and organizing data in such a way that we can perform operations on these data in an effective way. Data Structures is about rendering data elements in terms of some relationship, for better organization and storage. For example, we have some data which has, player's **name** "Virat" and **age** 34. Here "Virat" is of **String** data type and 34 is of **integer** data type.

We can organize this data as a record like **Player** record, which will have both player's name and age in it. Now we can collect and store player's records in a file or database as a data structure. **For example**: "Dhoni" 39, "Gambhir" 38, "Sehwag" 43

If you are aware of Object Oriented programming concepts, then a class also does the same thing, it collects different type of data under one single entity. The only difference being, data structures provides for techniques to access and manipulate data efficiently.

In simple language, Data Structures are structures programmed to store ordered data, so that various operations can be performed on it easily. It represents the knowledge of data to be organized in memory. It should be designed and implemented in such a way that it reduces the complexity and increases the efficiency.

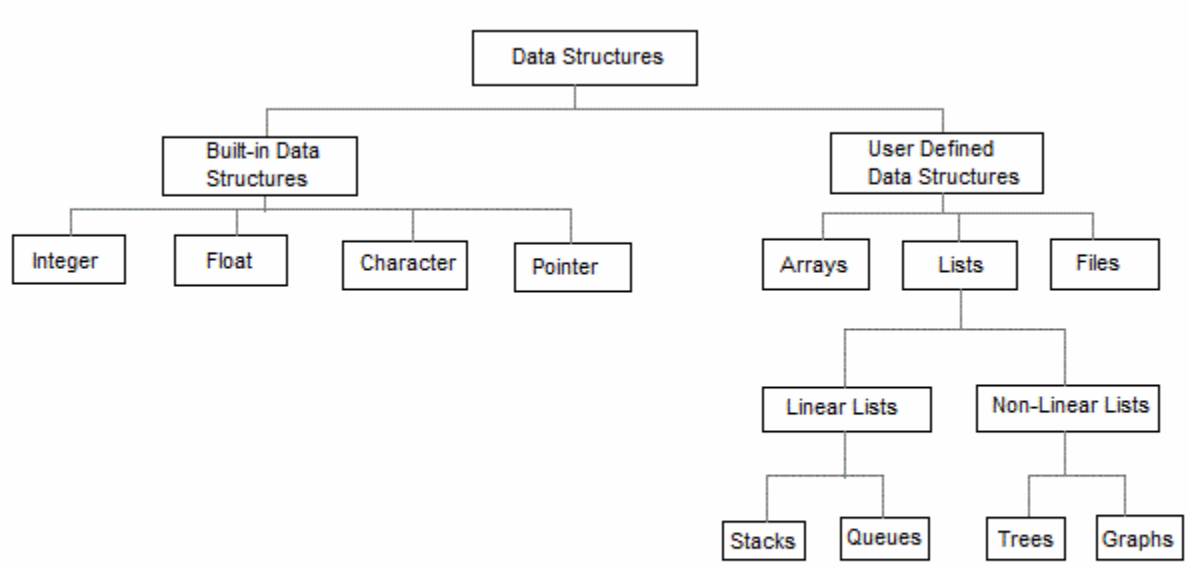
Basic types of Data Structures

As we have discussed above, anything that can store data can be called as a data structure, hence Integer, Float, Boolean, Char etc, all are data structures. They are known as **Primitive Data Structures**.

Then we also have some complex Data Structures, which are used to store large and connected data. Some example of **Abstract Data Structure** are :

- Linked List
- Tree
- Graph
- Stack, Queue etc.

All these data structures allow us to perform different operations on data. We select these data structures based on which type of operation is required. We will look into these data structures in more details in our later lessons.



The data structures can also be classified on the basis of the following characteristics:

Characteristic	Description
Linear	In Linear data structures, the data items are arranged in a linear sequence. Example: **Array**
Non-Linear	In Non-Linear data structures,the data items are not in sequence. Example: **Tree**, **Graph**
Homogeneous	In homogeneous data structures, all the elements are of same type. Example: **Array**
Non-Homogeneous	In Non-Homogeneous data structure, the elements may or may not be of the same type. Example: **Structures**
Static	Static data structures are those whose sizes and structures associated memory locations are fixed, at compile time. Example: **Array**
Dynamic	Dynamic structures are those which expand or shrink depending upon the program need and its execution. Also, their associated memory locations changes. Example: **Linked List created using pointers**

Data Abstract Data Type

Abstraction:

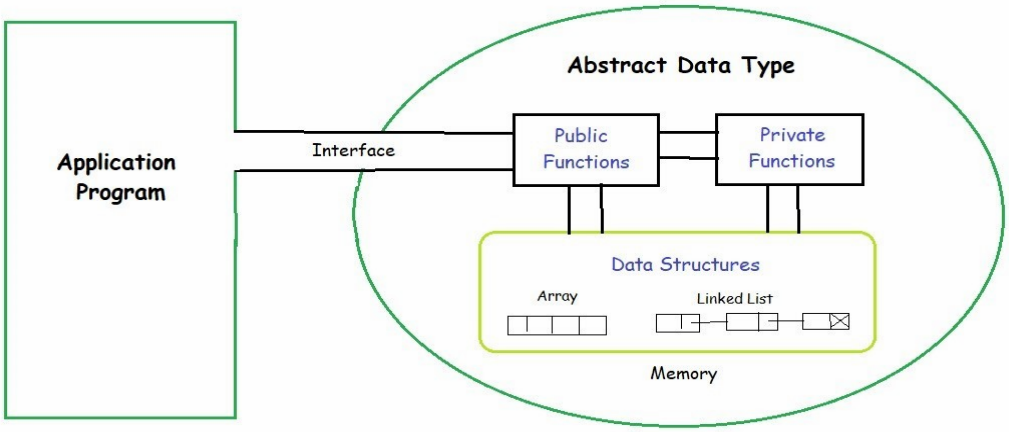
The first thing that we need to consider when writing programs is the *problem*. However, the problems that we asked to solve in real life are often nebulous or complicated. Thus we need to distill such as system down to its most fundamental parts and describe these parts in a simple, precise language. This process is called abstraction. Through abstraction, we can generate a model of the problem, which defines an abstract view of the problem. Normally, the model includes the data that are affected and the operations that are required to access or modify.

As an example, consider a program that manages the student records. The head of the Bronco Direct comes to you and asks you to create a program which allows to administer the students in a class. Well, this is too vague a problem. We need to think about, for example, what student information is needed for the record? What tasks should be allowed?

There are many properties we could think of about a student, such as name, DOB, SSN, ID, major, email, mailing address, transcripts, hair color, hobbies, etc. Not all these properties are necessary to solve the problem. To keep it simple, we assume that a student's record includes the following fields: (1) the student's name and (2) ID. The three simplest operations performed by this program include (1) adding a new student to the class, (2) searching the class for a student, given some information of the student, and (3) deleting a student who has dropped the class. These three operations can be furthermore defined as below:

- **ADD (stu_record)**: This operation adds the given student record to the collection of student records.
- **SEARCH (stu_record_id)**: This operation searches the collection of student records for the student whose ID has been given.
- **DELETE (stu_record_id)**: This operation deletes the student record with the given ID from the collection.

Now, we have modeled the problem in its most abstract form: listing the types of data we are interested in and the operations that we would like to perform on the data. We have not discussed anything about how these student records will be stored in memory and how these operations will be implemented. This kind of abstraction defines an ***abstract data type*** (**ADT**). The general structure of ADT is as shown in following figure.



Definition of ADT

An ADT is a mathematical model of a data structure that specifies the type of **data** stored, the **operations** supported on them, and the types of **parameters of the operations**. An ADT specifies what each operation does, but not how it does it. Typically, an ADT can be implemented using one of many different data structures. A useful first step in deciding what data structure to use in a program is to specify an ADT for the program.

In general, the steps of building ADT to data structures are:

1. Understand and clarify the nature of the target information unit.
2. Identify and determine which data objects and operations to include in the models.
3. Express this property somewhat formally so that it can be understood and communicate well.
4. Translate this formal specification into proper language. In C++, this becomes a .h file. In Java, this is called "user interface".
5. Upon finalized specification, write necessary implementation. This includes storage scheme and operational detail. Operational detail is expressed as separate functions (methods).

Data Structures: ADT and its Implementation

Now let us consider two alternate data structures for the above ADT: (1) **an unordered array** of records and (2) **an ordered array** of records, ordered by IDs. These different data structures greatly influence the implementation details and how fast and efficient the program runs.

First let us look at using an **unordered** array. Assume that the student records are stored in an array with no particular order. We can use a variable *n* to keep track of the number of students currently in the array.

- **ADD**: Simply take the record and store it in slot *n* of the array and increment *n*. This takes constant amount of time since the time is independent of *n*.
- **SEARCH**: Since the array is not ordered, we have to scan through the whole array to find the requested record. The result could be either the record is found or the record doesn't exist. The time taken to perform the search is proportional to *n* and the worst case

scenario is n. Of course, if the record I am looking for happens to be the fist item in the array, it only takes constant time. However, when determining the running time of an algorithm, we are often interested in worst case analysis.

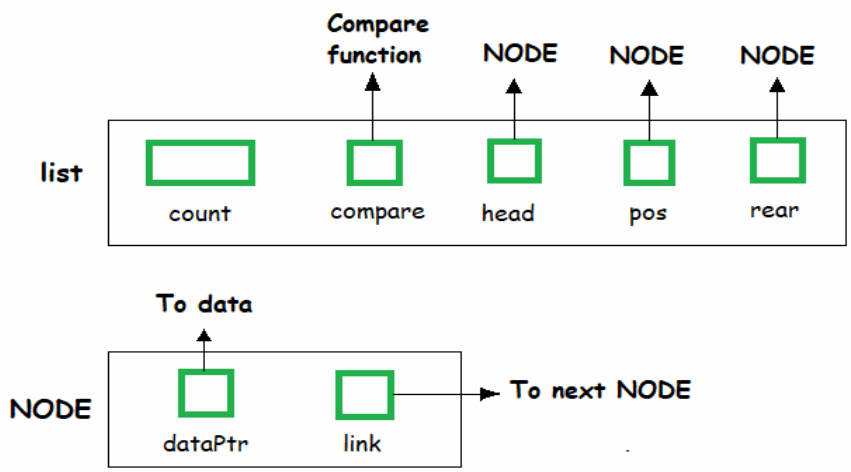
- **DELETE**: This operation requires us to first search for the given record. Once it is found, the algorithm can simply replace it by the last record and decrement n. Once the record is found, it only takes a constant amount of time to delete it. But time spent searching for the record is the same as above, proportional to n. Therefore, in all, this operation also takes time proportional to n, in the worst case.

Now let us consider using an **ordered** array. Assume that the student records are sorted in an array, with an increasing order of student IDs. We can also use a variable n to keep track of the number of students currently in the array.

- **ADD**: Since the array is sorted, we first need to find out where should we insert the record. This can be done by scanning through the array, comparing the current record in the array with the record we want to insert, and finding the smallest index i of the record whose ID is larger than the new ID. Then the new record should be put to the ith slot in the array. To do that, all the records in slots i, i+1, i+2, ..., n need to be moved down one slot to create an empty slot for the new record. We can then put the new record into the ith slot and increment n. This operation takes time proportional to n.
- **SEARCH**: Since the array of records is sorted by IDs, we can use a **binary search** to find the given record. In general, a binary search algorithm first compares the given ID with the ID in the middle of the array (with index $n/2$ or $(n+1)/2$). Then the algorithm branches based on different conditions: if the two IDs are the same, we find the record; if the given ID is smaller, the algorithm continues to search the first half of the current array, ignoring the second half; otherwise, the algorithm continues to search the second half of the current array, ignoring the first half. The operation time is $\log_2 n$. Finding the time for binary search will be discussed in the future.
- **DELETE**: This operation requires us to first search for the given record. This can be done in $\log_2 n$ time as shown above. Since the array is ordered, we need to fill in the empty slot i with a record, which means the records in slots i+1, i+2, ..., n need to be moved up one slot to cover the empty one. The time is also proportional to n. In all, this operation takes time proportional to n, in the worst case.

ADT for List

- The data is generally stored in key sequence in a list which has a head structure Consisting of *count*, *pointers* and *address of compare function* needed to Compare the data in the list.



- The data node contains the *pointer* to a data structure and a *self-referential pointer* which points to the next node in the list.

```
//List ADT Type Definitions
        typedef struct node
        {       void *DataPtr;
                struct node *link;
        } Node;
        typedef struct
        {       int count;
                Node *pos;
                Node *head;
                Node *rear;
                int (*compare) (void *argument1, void *argument2)
        } LIST;
```

Abstract data type for *Natural Number*

structure Natural_Number is

objects: an ordered subrange of the integers starting at zero and ending at the maximum integer (*INT_MAX*) on the computer

functions:

for all x, y ∈ *Nat_Number*; *TRUE, FALSE* ∈ *Boolean*
and where +, -, <, and == are the usual integer operations.

Nat_No Zero ( )	::= 0
Boolean Is_Zero(x)	::= **if** (x) **return** *FALSE* **else return** *TRUE*
Nat_No Add(x, y)	::= **if** ((x+y) <= *INT_MAX*) **return** x+y **else return** *INT_MAX*
Boolean Equal(x,y)	::= **if** (x== y) **return** *TRUE* **else return** *FALSE*
Nat_No Successor(x)	::= **if** (x == *INT_MAX*) **return** x **else return** x+1
Nat_No Subtract(x,y)	::= **if** (x<y) **return** 0 **else return** x-y

end *Natural_Number*

::= is defined as

Algorithm and its Efficiency Analysis

What is an Algorithm?

An algorithm is a finite set of instructions or logic, written in order, to accomplish a certain predefined task. Algorithm is not the complete code or program, it is just the core logic(solution) of a problem, which can be expressed either as an informal high level description as **pseudocode** or using a **flowchart**.

Every Algorithm must satisfy the following properties:

1. **Input**- There should be **0 or more inputs** supplied externally to the algorithm.
2. **Output**- There should be **at least 1 output** obtained.
3. **Definiteness**- Every step of the algorithm should be clear and well defined.
4. **Finiteness**- The algorithm should have finite number of steps.
5. **Correctness**- Every step of the algorithm must generate a correct output.

Example: Find Greatest Common Divisor of Two Nonnegative Numbers

Solution: one can get greatest common divisor for two numbers in three different ways like

1. ***Euclid's algorithm***

   gcd*(m n) = gcd(n, m mod n)*

   gcd*(60, 24) = gcd(24, 12) = gcd(12, 0) = 12.*

2. **Consecutive integer checking algorithm**

   For numbers 60 and 24, the algorithm will try first 24, then 23, and so on, until it reaches 12, where it stops.

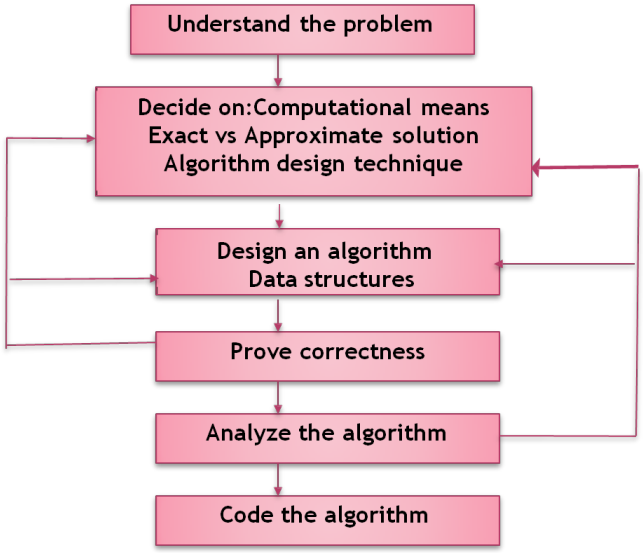
3. **Middle- School Procedure**

   For the numbers 60 and 24, we get 60 = **2 . 2 . 3** . *5* 24 = 2 . ***2 . 2 . 3***

   gcd*(60, 24) = 2 . 2 . 3 = 12.*

Same solution can be obtained by three different approves hence it is very important to understand the significance of every approach i.e need to do efficiency analysis for comparison of these methods and select most suitable and efficient approach for finding GCD of given numbers. In next part there is discussion about how to design an algorithm and how to do the efficiency analysis of algorithms.

Algorithm Design Stegs block diagram



1. Understand the problem

   - Understand completely the problem given
     - What will be the input?
     - What will be desirable Output?
   - An input to an algorithm specifies an instance of the problem which algorithm can solves.

2. Ascertaining the Capabilities of the Computational Device

   Need to analyze on which device algorithm may executed and computability of that device
   For Example:
   - Sequential Algorithms
     - Instructions are executed one after another, one operation at a time.
     - Random-access Machine (RAM)
   - Parallel Algorithms
     - Execute operations concurrently(parallel)

3. Choosing between Exact and Approximate Problem Solving

   - Exact algorithm
   - Approximation algorithm
     - Extracting square roots,
     - Solving nonlinear equations,
     - Evaluating definite integrals

4. Algorithm Design Technique

   - An algorithm design technique ("strategy" or "paradigm")
   - General approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.
   - Algorithm design techniques make it possible to classify algorithms according to an underlying design idea

5. Designing an Algorithm and Data Structures

   - Several techniques need to be combined,
   - Hard to pinpoint as applications of the known design techniques
   - Algorithms+ data structures = programs
   - Object-oriented programming Data structures remain crucially important for both design and analysis of algorithms.

6. Methods of Specifying an Algorithm

   - Pseudo code
     - A mixture of a natural language and programming language
   - Flowchart
     - A method of expressing an algorithm by a collection of connected geometric shapes containing descriptions of the algorithm's steps.
   - Programs

Different Approaches to Design an Algorithm:
An algorithm does not enforce a language or mode for its expression but only demands adherence to its properties.
Practical Algorithm Design Issues:

- To save time (**Time Complexity**): A program that runs faster is a better program.
- To save space (**Space Complexity**): A program that saves space over a competing program is considerable desirable.

Efficiency of Algorithms:

The performances of algorithms can be measured on the scales of time and space. The performance of a program is the amount of computer memory and time needed to run a program. We use two approaches to determine the performance of a program. One is analytical and the other is experimental. In performance analysis we use analytical methods, while in performance measurement we conduct experiments.
Time Complexity: The time complexity of an algorithm or a program is a function of the running time of the algorithm or a program. In other words, it is the amount of computer time it needs to run to completion.
Space Complexity: The space complexity of an algorithm or program is a function of the space needed by the algorithm or program to run to completion.
The time complexity of an algorithm can be computed either by an empirical or theoretical approach. The empirical or posteriori testing approach calls for implementing the complete algorithms and executing them on a computer for various instances of the problem. The time taken by the execution of the programs for various instances of the problem are noted and compared. The algorithm whose implementation yields the least time is considered as the best among the candidate algorithmic solutions.
Analyzing Algorithms
Suppose M is an algorithm, and suppose n is the size of the input data. Clearly the complexity f(n) of M increases as n increases. It is usually the rate of increase of f(n) with some standard functions. The most common computing times are
$O(1)$, $O(\log_2 n)$, $O(n)$, $O(n \log_2 n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$

Example:

Program Segment A

```
-------------------------
        x =x + 2;
-------------------------
```

Program Segment B

```
-------------------------
for k =1 to n do
        x =x + 2;
end;
-------------------------
```

Program Segment C

```
-------------------------
for j =1 to n do
  for x = 1 to n do
        x =x + 2;
  end
end;
-------------------------
```

Total Frequency Count of Program Segment A

Program Statements	Frequency Count
----------	
x =x + 2;	1

Total Frequency Count	1

Total Frequency Count of Program Segment B

Program Statements	Frequency Count
----------	
for k =1 to n do	(n+1)
x =x + 2;	n
end;	n

Total Frequency Count	3n+1

Total Frequency Count of Program Segment C

Program Statements	Frequency Count
----------	
for j =1 to n do	(n+1)
for x = 1 to n do	n(n+1)
x =x + 2;	n^2
end	n^2
end;	n

Total Frequency Count	$3n^2+3n+1$

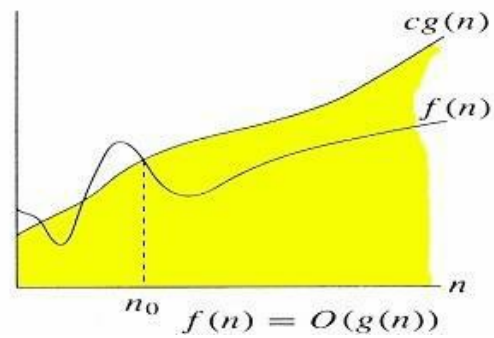
The total frequency counts of the program segments A, B and C given by 1, (3n+1) and ($3n^2+3n+1$) respectively are expressed as O(1), O(n) and O(n^2). These are referred to as the time complexities of the program segments since they are indicative of the running times of the program segments. In a similar manner space complexities of a program can also be expressed in terms of mathematical notations,

which is nothing but the amount of memory they require for their execution.

Asymptotic Notations:

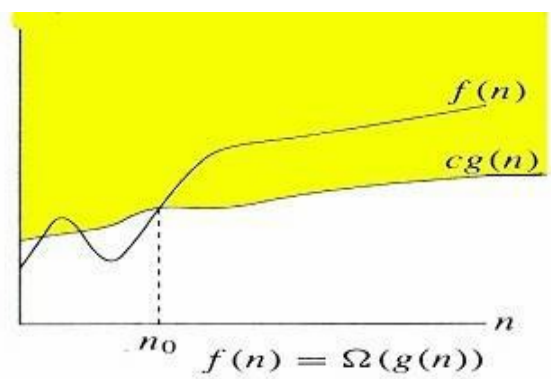
It is often used to describe how the size of the input data affects an algorithm's usage of computational resources. Running time of an algorithm is described as a function of input size n for large n.
Big oh(O): Definition: f(n) = O(g(n)) (read as f of n is big oh of g of n) if there exist a positive integer n_0 and a positive number c such that $|f(n)| \leq c|g(n)|$ for all $n \geq n_0$. Here g(n) is the upper bound of the function f(n).

f(n)	g(n)	
$16n^3 + 45n^2 + 12n$	n^3	$f(n) = O(n^3)$
$34n - 40$	n	$f(n) = O(n)$
50	1	$f(n) = O(1)$



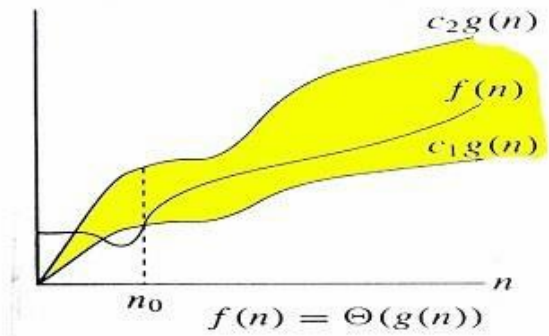
Omega(Ω): Definition: f(n) = Ω(g(n)) (read as f of n is omega of g of n), if there exists a positive integer n_0 and a positive number c such that $|f(n)| \geq c\,|g(n)|$ for all $n \geq n_0$. Here g(n) is the lower bound of the function f(n).

f(n)	g(n)	
$16n^3 + 8n^2 + 2$	n^3	$f(n) = \Omega(n^3)$
$24n + 9$	n	$f(n) = \Omega(n)$



Theta(Θ): Definition: f(n) = Θ(g(n)) (read as f of n is theta of g of n), if there exists a positive integer n_0 and two positive constants c_1 and c_2 such that $c_1\,|g(n)| \leq |f(n)| \leq c_2\,|g(n)|$ for all $n \geq n_0$. The function g(n) is both an upper bound and a lower bound for the function f(n) for all values of n, $n \geq n_0$.

f(n)	g(n)	
$16n^3 + 30n^2 - 90$	n^2	$f(n) = \Theta(n^2)$
$7.2^n + 30n$	2^n	$f(n) = \Theta(2^n)$



Little oh(o): Definition: f(n) = O(g(n)) (read as f of n is little oh of g of n), if f(n) = O(g(n)) and f(n) ≠ Ω(g(n)).

Time Complexity with growth rate of an algorithm

n	1	log n	n log n	n^2
1	1	0	0	1
2	1	1	2	4
4	1	2	8	16
8	1	3	24	64
16	1	4	64	256
32	1	5	160	1024
64	1	6	384	4096
128	1	7	896	16384
256	1	8	2048	65536
512	1	9	4608	262144
1024	1	10	10240	1048576
2048	1	11	22528	4194304
4096	1	12	49152	16777216
8192	1	13	106496	67108864
16384	1	14	229376	268435456

Growth rate

9

Time Complexities of various Algorithms:

Asymptotic Notations: Θ O Ω o ω

Rough, intuitive meaning worth remembering:

Theta	$f(n) = \Theta(g(n))$	$f(n) \approx c\ g(n)$
Big Oh	$f(n) = O(g(n))$	$f(n) \leq c\ g(n)$
Big Omega	$f(n) = \Omega(g(n))$	$f(n) \geq c\ g(n)$
Little Oh	$f(n) = o(g(n))$	$f(n) \ll c\ g(n)$
Little Omega	$f(n) = \omega(g(n))$	$f(n) \gg c\ g(n)$

Reasons for analyzing algorithms:

- To predict the resources that the algorithm requires
- Computational Time (CPU consumption).
- Memory Space (RAM consumption).
- Communication bandwidth consumption.
- To predict the running time of an algorithm
- Total number of primitive operations executed.

Recursive Algorithms:

GCD Design: Given two integers a and b, the greatest common divisor is recursively found using the formula

$$\gcd(a,b) = \begin{cases} a & \text{if } b=0 \\ b & \text{if } a=0 \end{cases}$$

General Case

Base case

gcd(b, a mod b)

Fibonacci Design: To start a fibonacci series, we need to know the first two numbers.

$$\text{Fibonacci}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{Fibonacci}(n-1) + \text{fibonacci}(n-2) & \end{cases}$$

Base case

General case

Difference between Recursion and Iteration:

1. A function is said to be recursive if it calls itself again and again within its body whereas iterative functions are loop based imperative functions.

2. Recursion uses stack whereas iteration does not use stack.

3. Recursion uses more memory than iteration as its concept is based on stacks.

4. Recursion is comparatively slower than iteration due to overhead condition of maintaining stacks.

5. Recursion makes code smaller and iteration makes code longer.

6. Iteration terminates when the loop-continuation condition fails whereas recursion terminates when a base case is recognized.

7. While using recursion multiple activation records are created on stack for each call where as in iteration everything is done in one activation record.

8. Infinite recursion can crash the system whereas infinite looping uses CPU cycles repeatedly.

9. Recursion uses selection structure whereas iteration uses repetetion structure.

Types of Recursion:

- Recursion is of two types depending on whether a function calls itself from within itself or whether two functions call one another mutually. The former is called direct recursion and the later is called indirect recursion. Thus there are two types of recursion:

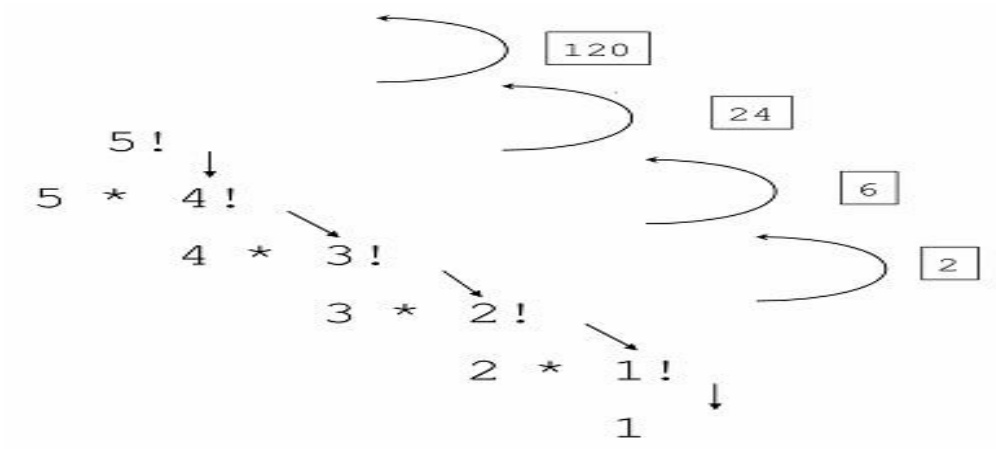
1. Direct Recursion

2. Indirect Recursion

- Recursion may be further categorized as:

- ✓ Linear Recursion
- ✓ Binary Recursion
- ✓ Multiple Recursion

Linear Recursion: It is the most common type of Recursion in which function calls itself repeatedly until base condition [termination case] is reached. Once the base case is reached the results are return to the caller function. If a recursive function is called only once then it is called a linear recursion.



Binary Recursion:

Some recursive functions don't just have one call to themselves; they have two (or more). Functions with two recursive calls are referred to as binary recursive functions.

Example1: The Fibonacci function fib provides a classic example of binary recursion. The Fibonacci numbers can be defined by the rule:

fib(n) = 0 if n is 0,
= 1 if n is 1,
= fib(n-1) + fib(n-2) otherwise

For example, the first seven Fibonacci numbers are

Fib(0) = 0
Fib(1) = 1

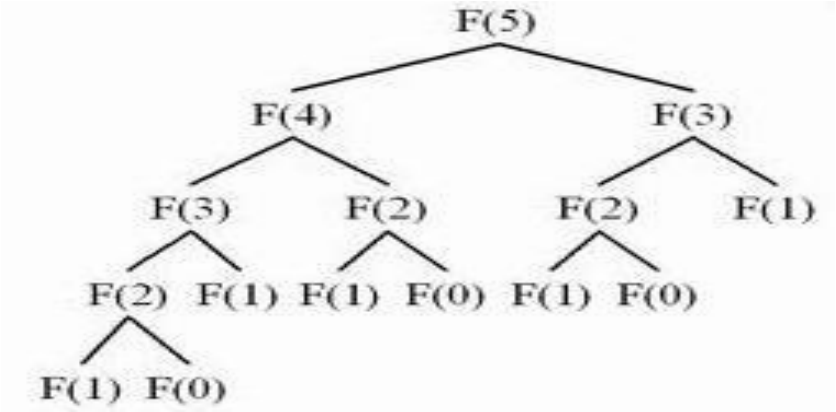
Fib(2) = Fib(1) + Fib(0) = 1

Fib(3) = Fib(2) + Fib(1) = 2

Fib(4) = Fib(3) + Fib(2) = 3

Fib(5) = Fib(4) + Fib(3) = 5

Fib(6) = Fib(5) + Fib(4) = 8



Program to display the Fibonacci sequence up to n-th term where n is provided by the user

change this value for a different result

```
nterms = 10
uncomment to take input from the user #nterms = int(input("How many terms? "))
first two terms
        n1 = 0
        n2 = 1
count = 0
check if the number of terms is valid if nterms <= 0:
        print("Please enter a positive integer") elif nterms == 1:
        print("Fibonacci sequence upto",nterms,":") print(n1)
else:
        print("Fibonacci sequence upto",nterms,":")
while count < nterms:
        print(n1,end=' , ')
        nth = n1 + n2
        update values n1 = n2
        n2 = nth count += 1
```

An algorithm is said to be efficient and fast, if it takes less time to execute and consumes less memory space. The performance of an algorithm is measured on the basis of following properties :

1. Time Complexity
2. Space Complexity

Space Complexity

Its the amount of memory space required by the algorithm, during the course of its execution. Space complexity must be taken seriously for multi-user systems and in situations where limited memory is available.

An algorithm generally requires space for following components :

- **Instruction Space:** It's the space required to store the executable version of the program. This space is fixed, but varies depending upon the number of lines of code in the program.
- **Data Space:** Its the space required to store all the constants and variables (including temporary variables) value.
- **Environment Space:** Its the space required to store the environment information needed to resume the suspended function.

Space Complexity $S(P) = C + S_P(I)$

- Fixed Space Requirements (C)
  Independent of the characteristics of the inputs and outputs
  - ✓ Instruction space
  - ✓ Space for simple variables, fixed-size structured variable, constants
- Variable Space Requirements ($S_P(I)$)
  depend on the instance characteristic I
  - ✓ Number, size, values of inputs and outputs associated with I
  - ✓ Recursive stack space, formal parameters, local variables, return address

While executing, algorithm uses memory space for three reasons:

1. **Instruction Space**

   It's the amount of memory used to save the compiled version of instructions.

2. **Environmental Stack**

   Sometimes an algorithm(function) may be called inside another algorithm(function). In such a situation, the current variables are pushed onto the system stack, where they wait for further execution and then the call to the inside algorithm(function) is made.

   For example, If a function A() calls function B() inside it, then all th variables of the function A() will get stored on the system stack temporarily, while the function B() is called and executed inside the funciton A().

3. **Data Space**

   Amount of space used by the variables and constants.

But while calculating the **Space Complexity** of any algorithm, we usually consider only **Data Space** and we neglect the **Instruction Space** and **Environmental Stack**.

Calculating the Space Complexity

For calculating the space complexity, we need to know the value of memory used by different type of data type variables, which generally varies for different operating systems, but the method for calculating the space complexity remains the same.

Type	Size
bool, char, unsigned char, signed char, __int8	1 byte
int16, short, unsigned short	2 bytes
float, __int32, int, unsigned int, long, unsigned long	4 bytes
double, __int64, long double, long long	8 bytes

Example 1: Simple arithmetic function

```
float abc (float a, float b, float c)
{
   return a + b + b * c + (a + b - c) / (a + b) + 4.00;
 }
```

$S_{sum}(I) = 0$

Example 2: Iterative function for summing a list of numbers

```
float sum (float list[ ], int n)
{
  float tempsum = 0;
  int i;
  for (i = 0; i<n; i++)
     tempsum += list [i];
  return tempsum;
}
```

$S_{sum}(I) = 0$

Recall: pass the address of the first element of the array & pass by value

Example 3: Recursive function for summing a list of numbers

```
float rsum(float list[ ], int n)
{
   if (n) return rsum(list, n-1) + list[n-1];
   return 0;
 }
```

$S_{sum}(I)=S_{sum}(n)=6n$

***Figure 1.1:** Space needed for one recursive call of Program

Type	Name	Number of bytes
parameter: float parameter: integer return address:(used internally)	list [ ] n	2 2 2(unless a far address)
TOTAL per recursive call		6

Algorithmic Strategies

What is an algorithm?

An Algorithm is a procedure to solve a particular problem in a finite number of steps for a finite-sized input. The algorithms can be classified in various ways. They are:

1. Implementation Method
2. Design Method
3. Other Classifications

Classification by Implementation Method: There are primarily three main categories into which an algorithm can be named

1. **Recursion or Iteration:** A recursive algorithm is an algorithm which calls itself again and again until a base condition is achieved whereas iterative algorithms use loops and/or data structures like stacks, queues to solve any problem. Every recursive solution can be implemented as an iterative solution and vice versa.
   Example: The Tower of Hanoi is implemented in a recursive fashion while Stock Span problem is implemented iteratively.
2. **Exact or Approximate:** Algorithms that are capable of finding an optimal solution for any problem are known as the exact algorithm. For all those problems, where it is not possible to find the most optimized solution, an approximation algorithm is used. Approximate algorithms are the type of algorithms that find the result as an average outcome of sub outcomes to a problem.
   Example: For NP-Hard Problems, approximation algorithms are used. Sorting algorithms are the exact algorithms.
3. **Serial or Parallel or Distributed Algorithms:** In serial algorithms, one instruction is executed at a time while parallel algorithms are those in which we divide the problem into subproblems and execute them on different processors. If parallel algorithms are distributed on different machines, then they are known as distributed algorithms.

Classification by Design Method: There are primarily three main categories into which an algorithm can be named and They are:

1. **Divide and Conquer:** The Divide and Conquer strategy involves dividing the problem into sub-problem, recursively solving them, and then recombining them for the final answer.
   Example: Merge sort, Quicksort.
2. **Greedy Method:** In the greedy method, at each step, a decision is made to choose the *local optimum*, without thinking about the future consequences.
   Example: Fractional Knapsack, Activity Selection.
3. **Dynamic Programming:** The approach of Dynamic programming is similar to divide and conquer. The difference is that whenever we have recursive function calls with the same result, instead of calling them again we try to store the result in a data structure in the form of a table and retrieve the results from the table. Thus, the overall time complexity is reduced. "Dynamic" means we dynamically decide, whether to call a function or retrieve values from the table.
   Example: 0-1 Knapsack, subset-sum problem.
4. **Linear Programming:** In Linear Programming, there are inequalities in terms of inputs and maximizing or minimizing some linear functions of inputs.
   Example: Maximum flow of Directed Graph
5. **Reduction(Transform and Conquer):** In this method, we solve a difficult problem by transforming it into a known problem for which we have an optimal solution. Basically, the goal is to find a reducing algorithm whose complexity is not dominated by the resulting reduced algorithms.
   Example: Selection algorithm for finding the median in a list involves first sorting the list and then finding out the middle element in the sorted list. These techniques are also called *transform and conquer.*

Other Classifications: Apart from classifying the algorithms into the above broad categories, the algorithm can be classified into other broad categories like:

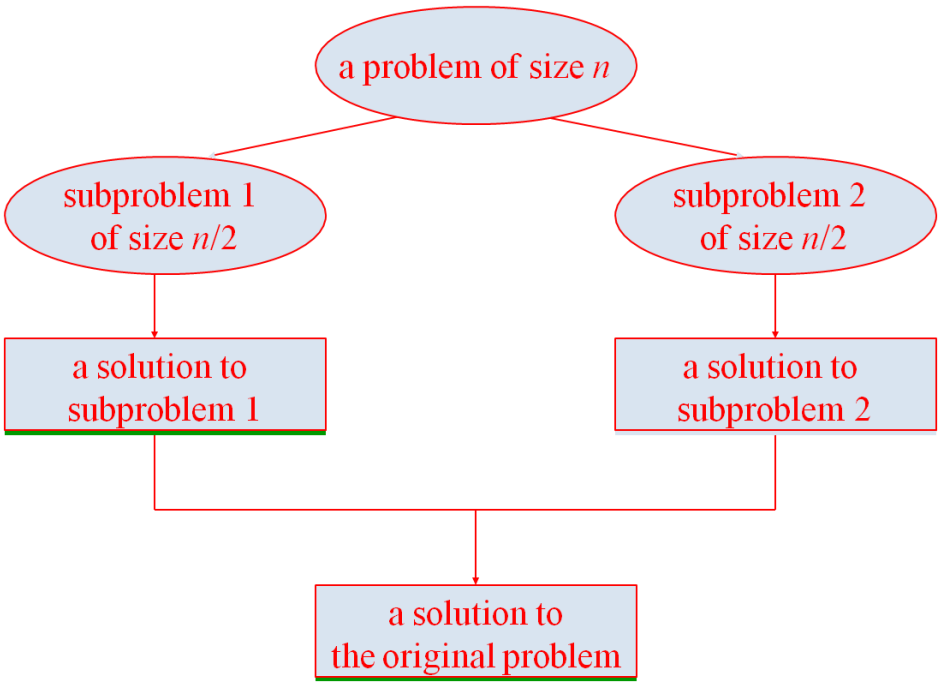
1. **Randomized Algorithms:** Algorithms that make random choices for faster solutions are known as randomized algorithms.
   Example: Randomized Quick sort Algorithm.
2. **Classification by complexity:** Algorithms that are classified on the basis of time taken to get a solution to any problem for input size. This analysis is known as time complexity analysis.
   Example: Some algorithms take O(n), while some take exponential time.
3. **Classification by Research Area:** In CS each field has its own problems and needs efficient algorithms.
   Example: Sorting Algorithm, Searching Algorithm, Machine Learning etc.
4. **Branch and Bound Enumeration and Backtracking:** These are mostly used in Artificial Intelligence.

Attention reader! Don't stop learning now. Get hold of all the important DSA concepts with the **DSA Self Paced Course** at a student-friendly price and become industry ready. To complete your preparation from learning a language to DS Algo and many more, please refer **Complete Interview Preparation Course.**

Algorithm Design Techniques

1. **Divide and Conquer**

- Divide the problem into a number of subproblems
- Conquer the subproblems (solve them)
- Combine the sub problem solutions to get the Solution to the original problem
- Note: often the "conquer" step is done recursively
- A general methodology for using recursion to design efficient algorithms
- It solves a problem by:
    - Diving the data into parts
    - Finding sub solutions for each of the parts
    - Constructing the final answer from the sub solutions



- **Based on dividing problem into subproblems**

  Divide problem into sub-problems Subproblems must be of same type Sub problems do not need to overlap

  Conquer by solving sub-problems recursively. If the sub-problems are small enough, solve them in brute force fashion

  Combine the solutions of sub-problems into a solution of the original problem (tricky part)

- For Divide-and-Conquer algorithms the running time is mainly affected by 3 criteria:
- The number of sub-instances into which a problem is split.
- The ratio of initial problem size to sub- problem size.
- The number of steps required to divide the initial instance and to combine sub- solutions.
- **Algorithm for General Divide and Conquer Sorting**
  - Begin Algorithm

    Start Sort (L)

    If L has length greater than 1 then Begin

    Partition the list into two lists, high and low Start Sort(high)

    Start Sort (low) Combine high and low

    End
  - End Algorithm